

ACTIVITY 7 WALKTHROUGH

*I was thinking should I make this short walkthrough as a read.me file in my python scripts but I thought that this (PDF) is a better way to implement walkthrough because on this way I can show you screenshots of interface so that it is easier to navigate through app rather than plain text. Also while implementing all scripts, I tried to follow the speeches I sent to you via mail as much as I could!

1. HOW TO RUN SCRIPT LOCALLY

This document accompanies the ZIP file submitted for Activity 7. The ZIP contains four main folders: simulation (Python script), data (input CSV), output (generated JSON and CSV), and interface (HTML, CSS and JS files).

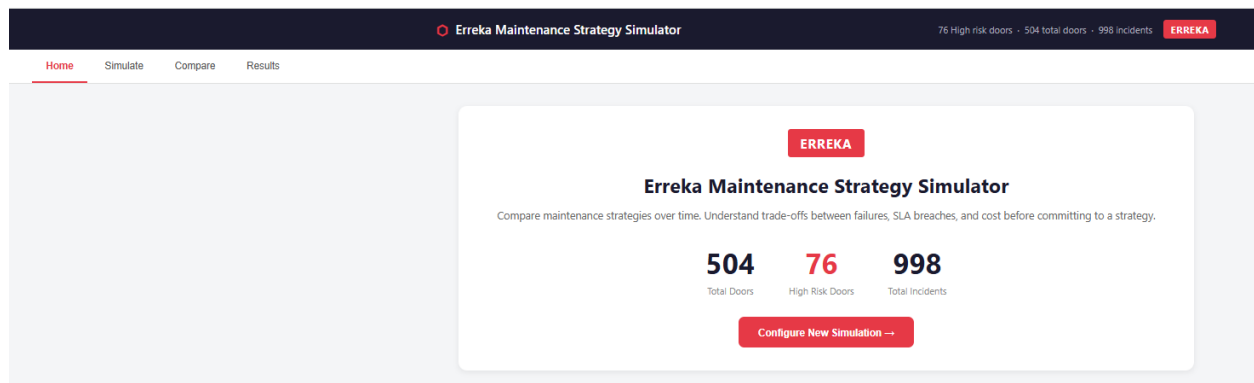
The output folder already contains the simulation results from my run, so you can open the application directly without any additional steps. To do so, navigate to the interface folder in VS Code, right-click on index.html, and select Open with Live Server. The application will open automatically in your browser. Note that opening index.html by double-clicking will not work due to CORS restrictions — Live Server is required.

If you want to generate fresh results, simply delete the two files inside the output folder (simulation_results.json and simulation_results.csv), then open a terminal in VS Code, navigate to the simulation folder, and run:

```
py -3.13 erreka_simulation.py
```

The script will automatically create two new files in the output folder and print the full simulation output in the terminal.

*files will be new but their content is the same as it is in the ones that I have put in zip file!



Erreka Maintenance Strategy Simulator – home screen

The home screen is the entry point of the application. It displays the Erreka logo and the name of the simulator in the header, along with a quick summary of the current installed base — 504 total doors, 76 high risk doors, and 998 total incidents. These numbers are taken directly from the data generated in Activities 3 and 5.

The navigation bar at the top allows moving between the four main sections of the application: Home, Simulate, Compare, and Results.

The only action available from this screen is clicking the Configure New Simulation button, which takes the user to the scenario configuration screen where the simulation parameters are set.

Scenario Configuration ERREKA

Strategy selector

☒ Reactive only ☒ Preventive (usage based) ☒ Risk & SLA based

Time horizon

☐ 3 months ☒ 6 months ☐ 1 year

Filters

☒ Door type (ROLLfast / ECOline) ☒ Country ☒ Criticality level

[View Strategy Rules & Run →](#)

Erreka Maintenance Strategy Simulator – Simulate screen

The Simulate screen is where the user configures the scenario before running anything. I designed it as a single, simple form — three groups of options, one button.

Strategy selector. The user picks which maintenance strategies they want to compare. All three are enabled by default since comparing them is the whole point of the DSS, but any can be deselected — for example, if the user already knows Reactive is unacceptable and only wants to compare Preventive against Risk & SLA.

Time horizon. The user chooses 3 months, 6 months, or 1 year. The default is 6 months: long enough to see meaningful patterns, short enough to be tactically relevant. The Python script always runs the full 12-month simulation, and the interface slices the results to the chosen horizon — so the user can switch views instantly without re-running anything.

Filters. The interface exposes three filtering dimensions — door type, country, and criticality level — that mirror those used in Activities 4 and 5. In this version they are present in the UI but not yet wired to the simulation engine, which still runs across the full installed base of 504 doors. I left the controls visible because the filtering logic is already defined in the previous activities and only needs to be connected to the simulation loop. It's a planned extension rather than a missing feature.

The "View Strategy Rules & Run →" button doesn't run the simulation immediately. It first takes the user through two explanatory screens — the Strategy Rules table and the Engine Flow — so they understand the model before seeing the output. A DSS that just spits out numbers without explaining the engine isn't useful for real decision-making.

Strategy Rules Table **ERREKA**

	Reactive	Preventive	Risk & SLA
Trigger	After fail	Usage $\geq$ 1000/mo	Risk score $\geq$ 2
Inspection frequency	None	Monthly	Based on score
Repair speed	Slow	Medium	Fast (critical)
Cost per action	High	Medium	Medium-high

* Rules are applied at each monthly time step to determine transition probabilities. These are modelling assumptions, not empirically validated values.

← Back

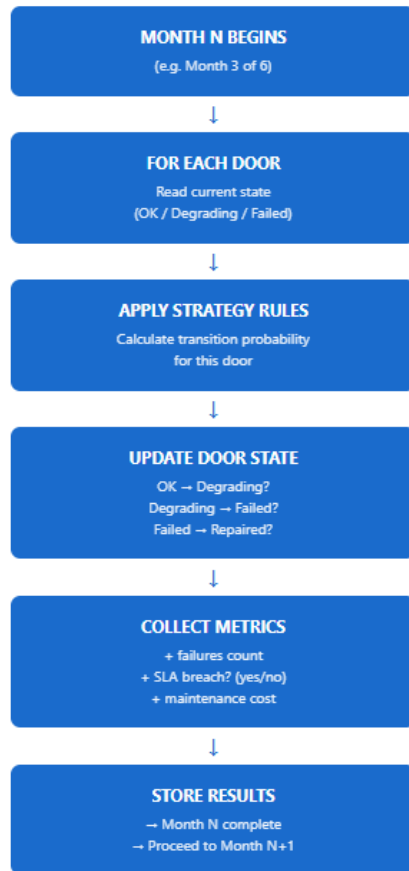
Run Simulation →

The Strategy Rules table is the first explanatory screen after pressing "View Strategy Rules & Run". It makes the differences between the three strategies concrete before the simulation runs, so the results later on are not a black box.

Each row captures one operational dimension. Trigger defines when a strategy intervenes — Reactive only after a failure, Preventive on a usage threshold, Risk & SLA whenever a door's risk score crosses a limit. Inspection frequency captures the proactive workload: Reactive does none, Preventive checks every door monthly, Risk & SLA visits high-risk doors more often than safe ones. Repair speed reflects how fast a failure gets resolved, which depends on whether the team is already on site. Cost per action summarises the financial profile: Reactive is expensive per incident, Preventive has lower per-action cost but high total cost because every door is inspected, and Risk & SLA sits in the middle with targeted interventions.

The note at the bottom is critical: these rules drive the transition probabilities of the simulation. They are modelling assumptions justified by the logic of each strategy, not values learned from historical Erreka data — exactly as the professor's feedback requested.

What Happens at Each Time Step? ERREKA



** This loop runs for each of the 504 doors, for each month, for each strategy — 3 parallel simulations.*

← Back

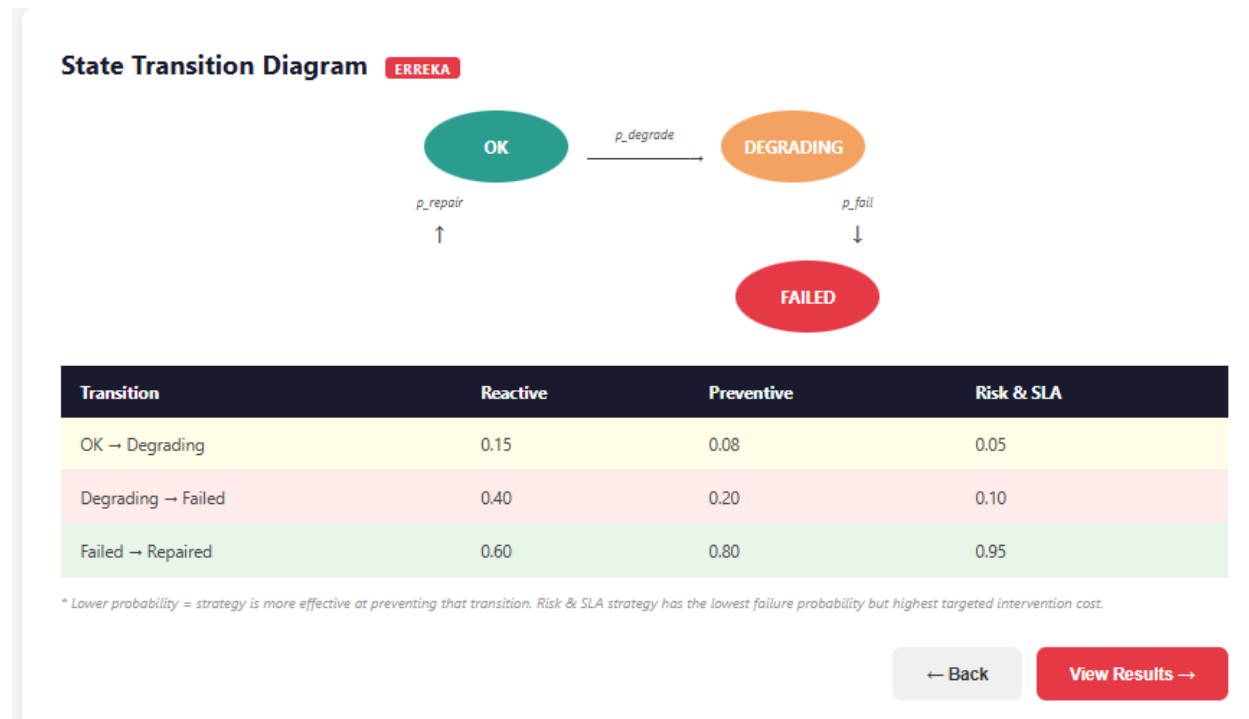
View State Transitions →

The Engine Flow screen opens up the simulation loop so the user can see exactly what happens inside each monthly time step. I added this screen because the professor's feedback specifically pointed out that the complexity of the project lies in how states evolve, how policies are applied, and how transitions are calculated — and a chart of results doesn't show any of that.

The loop reads top to bottom. Each month begins, the engine iterates through all 504 doors, reads each door's current state (OK, Degrading, or Failed), and applies the rules of the chosen strategy to compute a transition probability for that specific door. The state is then updated stochastically based on that probability — a door may degrade, fail, or be repaired. The engine collects the month's metrics (failure count, SLA breach yes/no, maintenance cost), stores them, and moves on to the next month.

The note at the bottom is important: this loop runs three times in parallel, once per strategy. Every door is simulated under all three policies starting from the same initial state, which is what makes

the final comparison fair. If each strategy ran on a different starting point, the differences between them would mean nothing.

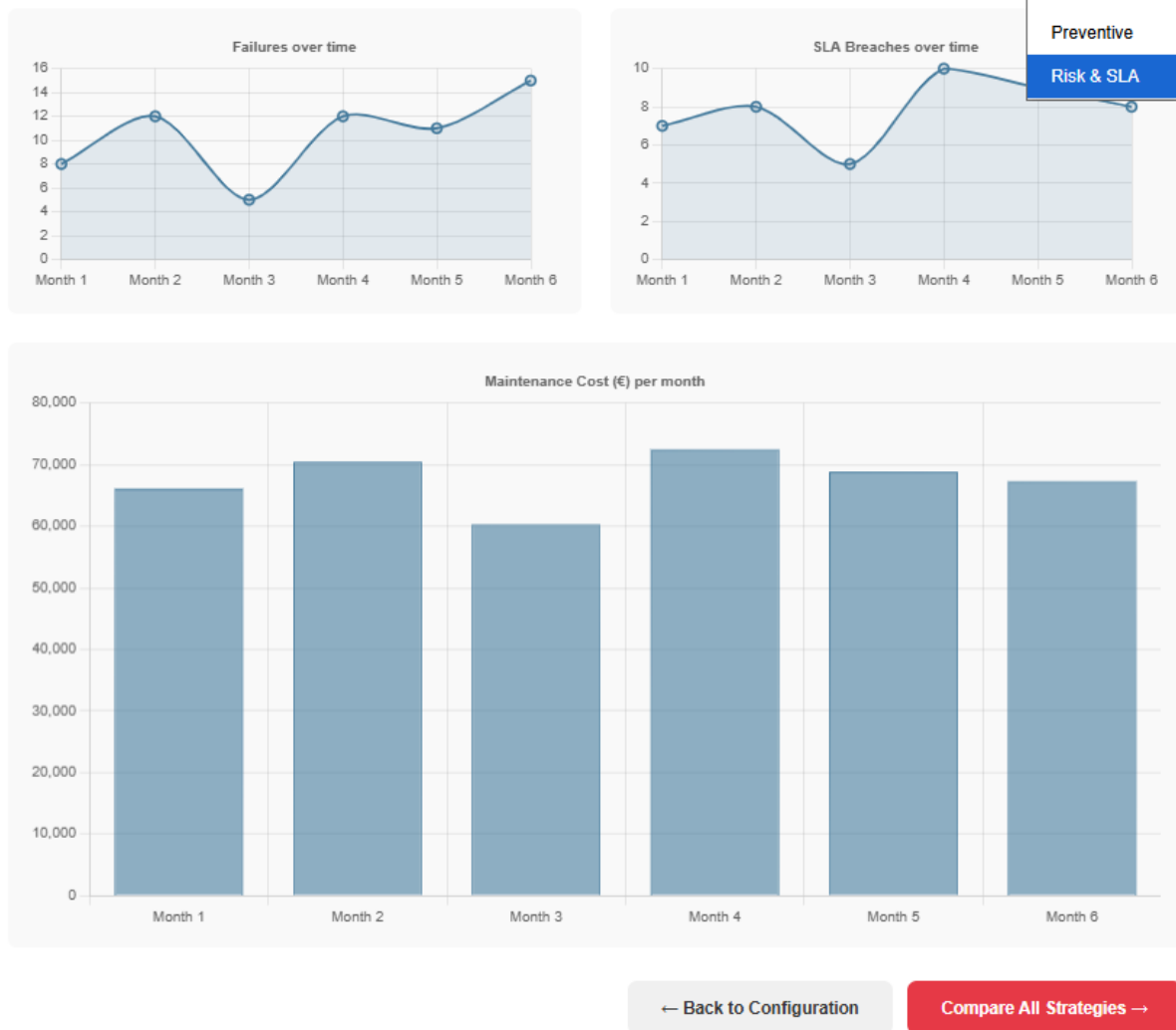


The State Transition Diagram is where the simulation's mathematical core becomes visible. Every door in the model lives in one of three states — OK, Degrading, or Failed — and moves between them each month with a probability that depends on the active strategy. The diagram at the top shows the allowed transitions: a healthy door can start degrading, a degrading door can fail, and a failed door can be repaired back to OK. There is no direct OK → Failed transition, because real doors don't break without warning; they wear down first.

The table below gives the actual numbers used by the simulation. Reading down each column tells the story of the strategy. Reactive lets degradation happen freely (0.15 chance per month) and a degrading door has a 40% chance of failing — because nobody is watching. Preventive cuts these in half thanks to scheduled inspections that catch problems early. Risk & SLA pushes them even lower because it concentrates resources on the doors that are most likely to fail in the first place. The Failed → Repaired row works in reverse: a higher number is better, because it means the strategy gets failed doors back online faster. Risk & SLA reaches 0.95 because critical-SLA doors are prioritised explicitly.

These values are modelling assumptions, justified conceptually by the logic of each strategy rather than derived from Erreka's historical data. I made this explicit on every screen where probabilities appear, because the professor specifically flagged this point in his feedback — and because a DSS that hides its assumptions is worse than one that admits them.

Results: Single Strategy View



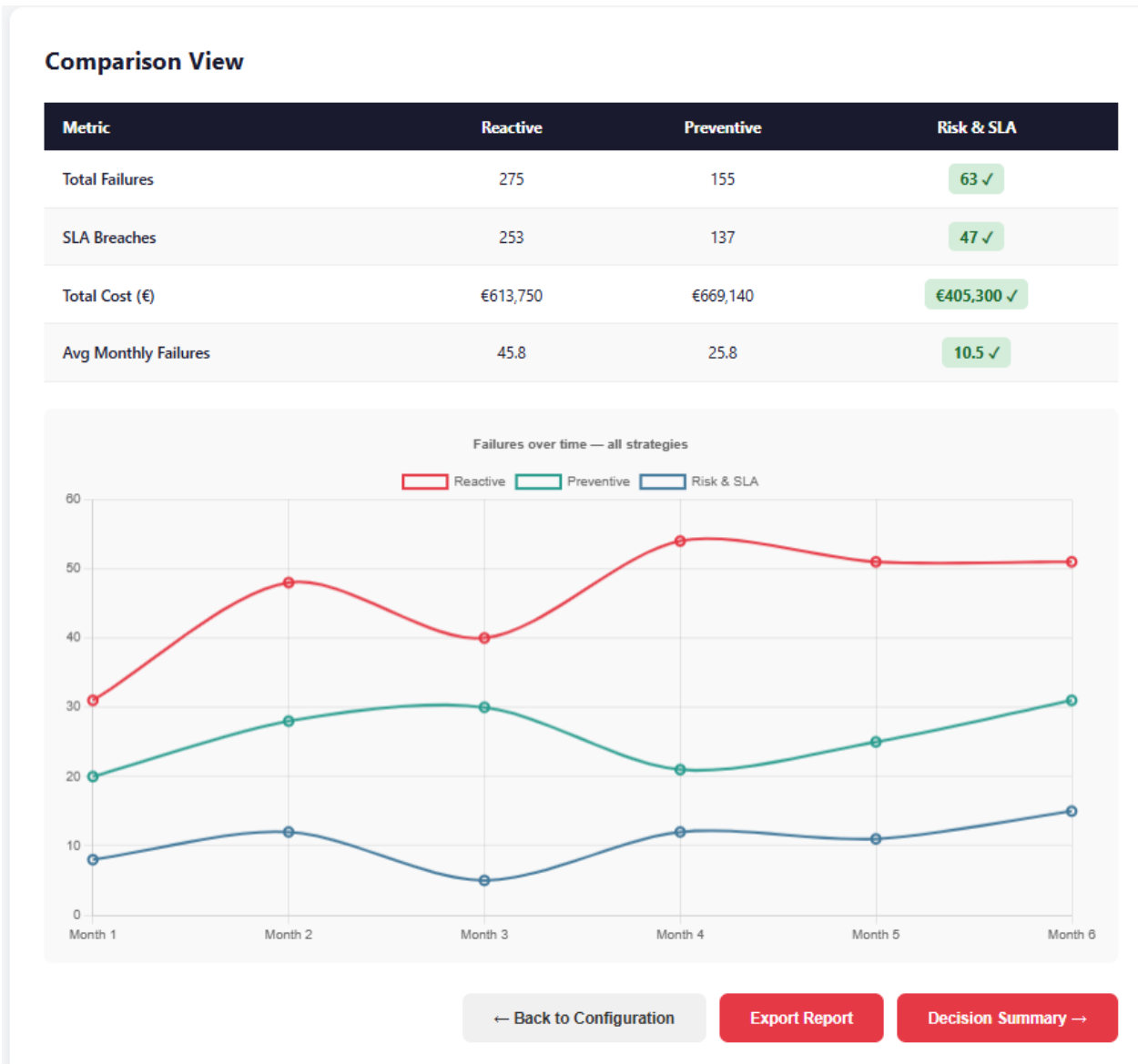
The Single Strategy View is the first screen where the user actually sees simulation results. I designed it as a deep-dive into one strategy at a time, before the user moves on to side-by-side comparison — because looking at three strategies overlaid at once is overwhelming if you haven't first understood what one of them looks like on its own.

The dropdown in the top right lets the user switch between the three strategies without leaving the screen. Each switch instantly redraws all three charts with the data of the selected strategy and colours them consistently across the whole app — red for Reactive, teal for Preventive, blue for Risk & SLA — so the user always knows which strategy they are looking at, even on a quick glance.

The three charts show different angles of the same simulation. Failures over time and SLA breaches over time are line charts because they capture how the strategy performs month by month — whether it stabilises, gets worse, or oscillates. Maintenance cost is a bar chart because monthly cost is a discrete budget number, not a continuous trend. Reading them together is the point: a

strategy might look good on failures but spike on cost, or vice versa, and the user needs to see all three to judge whether the trade-off is acceptable.

The charts respect the time horizon chosen on the Configure screen. If the user picked 6 months, only the first 6 months of the underlying 12-month simulation are shown — the same slicing logic explained earlier.



The Comparison View is the screen where the actual decision starts to take shape. Instead of looking at strategies one by one, the user now sees all three side by side on the same metrics and the same time axis. This is the core of a decision support system: making trade-offs visible at a glance.

The table at the top summarises the four headline metrics for the chosen time horizon — total failures, SLA breaches, total cost, and average monthly failures. For each row the best value is highlighted in green with a checkmark, so the user can spot the winning strategy on each dimension

without doing the arithmetic themselves. In this run Risk & SLA wins all four, but that won't always be the case once the filters are wired in and different door subsets are analysed — for some segments Preventive could be the better choice on cost, for instance.

The line chart below shows failures over time for all three strategies on the same axes. Reading it is more informative than the table alone: it shows not just totals but also the trajectory. Reactive drifts upward as unrepaired doors accumulate, Preventive stays roughly flat thanks to constant inspections, and Risk & SLA stays low and stable because it intervenes early on the riskiest doors. The shape of the curves is what tells the user whether a strategy is sustainable in the long run, or whether it just looks good in the short term.

The "Export Report" button generates a plain-text summary of the run — strategy totals plus a month-by-month breakdown — so the user can keep a record of the comparison outside the application. The "Decision Summary →" button leads to the final screen, where the simulation results are translated into an actionable recommendation.

Decision Summary / Recommendation

ERREKA

Based on your simulation, the Risk & SLA-based strategy reduces total failures by 75% and SLA breaches by 80% compared to the Reactive strategy, with a cost saving of €354,400 over the simulated period. The Preventive strategy reduces failures compared to Reactive but results in the highest total cost (€1,253,040) due to regular inspection overhead across all 504 doors. The Risk & SLA strategy delivers the best outcome across all three key metrics.

Top 5 doors requiring immediate intervention

Preventive

Reactive

Preventive

Risk & SLA

#	Door ID	Risk	SLA	Details
1	I-IT-000130	High risk	Critical SLA	ROLLfast · Italy · 5 failures
2	I-BR-000478	High risk	Critical SLA	ROLLfast · Brazil · 4 failures
3	P-MX-000436	High risk	Critical SLA	ECOLine Ped. · Mexico · 4 failures
4	I-MX-000445	High risk	Critical SLA	ROLLfast · Mexico · 4 failures
5	P-CN-000273	High risk	Critical SLA	ECOLine Ped. · China · 4 failures

Preventive strategy: only doors that are NOT currently failing are listed. Ranked by usage intensity and historical failures — these consume the inspection cycle fastest.

Start New Simulation →

The Decision Summary is the final screen and the most important one from a DSS perspective. Everything else in the application builds up to this moment — the user has configured the scenario, seen the rules, watched the simulation run, examined each strategy individually and compared them side by side. Now the system has to translate all of that into something the user can act on.

The blue box at the top is the textual recommendation. It is generated dynamically from the simulation results, not hardcoded — the percentages, savings, and totals all come from the actual numbers the engine produced for the selected horizon. If the user changes the configuration and

reruns, this text changes with it. The recommendation always frames the comparison against Reactive, because Reactive is the implicit baseline most maintenance teams start from, and the value of Preventive or Risk & SLA only becomes meaningful as a delta against doing nothing.

The Top 5 doors table is the most concrete output of the entire DSS. It answers the question a maintenance planner actually asks at the end of the analysis: "OK, but which doors do I intervene on first?" The dropdown on the right lets the user toggle the ranking by strategy, and this is the part I want to highlight — the list genuinely changes depending on the chosen strategy, because each strategy applies its own logic to define "priority". Reactive only lists doors that are currently in failure. Preventive ignores those and instead surfaces high-usage doors that are still operational but likely to fail next. Risk & SLA ranks by business impact: risk score combined with SLA criticality, regardless of current state. The explanatory note below the table changes with the dropdown so the user always knows why these specific doors were chosen.

This is what closes the loop on the DSS. The simulation produces aggregate metrics, the comparison view makes the trade-off visible, and the Decision Summary turns that trade-off into a ranked, named list of actions. Without this last step the user would have charts but no decision — and a decision support system that doesn't end in a decision isn't doing its job.

DSS VALUE

The question that drove this activity is simple: what decision does this system actually help someone make, and who is that someone?

The user I had in mind is a maintenance planner at Erreka — someone whose job is to decide how the company should allocate its maintenance budget across an installed base of 504 doors spread over multiple countries. This person already has access to descriptive analytics from Activity 4 (what failures happened, where, and to which doors) and to the risk classification from Activity 5 (which doors are most likely to fail next). What they did not have is a way to answer the next question: given everything we know about the installed base, which maintenance strategy should we actually adopt?

This is a fundamentally different kind of question. Descriptive analytics tells you what happened. Predictive analytics tells you what is likely to happen. Neither of them tells you what to do about it. The strategy choice is prescriptive — it requires reasoning about future scenarios under different policies, and the trade-offs between them. Until now, that decision would have been made on intuition and experience: someone in a meeting would argue for switching to preventive maintenance, someone else would push back on the cost, and the discussion would stall because nobody had the numbers to settle it.

This is the gap the Activity 7 DSS fills. It lets the planner take the same installed base and project it forward in time under three different policies — Reactive, Preventive, and Risk & SLA — and see, with concrete numbers, what each one produces in terms of failures, SLA breaches, and total maintenance cost. The user doesn't have to believe the model on faith: every screen explains the rules, the transitions, and the assumptions, so the planner can challenge the numbers if they disagree with the underlying logic.

The value of the system is not in the specific numbers produced by this simulation run. The transition probabilities are modelling assumptions, not values learned from Erreka's real failure history, and the absolute totals would shift once those probabilities are recalibrated on historical data. The value is in the structure: a repeatable way to ask "what would happen if we did X instead of Y", a comparison framework that exposes trade-offs across multiple metrics at the same time, and a final recommendation that translates aggregate results into a concrete list of doors to intervene on first.

This is also what makes the system different from the dashboards built in earlier activities. A dashboard tells the user the current state of the world; this DSS lets the user imagine alternative versions of the future and compare them before committing to any of them. That shift — from observation to projection, from data to decision — is what the activity was asking for.

The natural next step is to feed the simulation engine with real Erreka failure data instead of assumed probabilities, and to wire up the filters so the planner can run the same analysis on subsets of the installed base — for instance, "what is the best strategy for ROLLfast doors in countries with Critical SLAs?" The architecture is ready for that. The conceptual gap between the current version and a fully production-ready DSS is no longer about design — it's about data.